STAT GR5398 · COLUMBIA UNIVERSITY · FALL 2026

ASSIGNMENT 2 DESIGN BRIEF

Assignment 2: Selection, Valuation and Timing, Combined into a Backtestable Portfolio

Selection is provided by the course (the course stock-selection model, with readable source; students can retrain it for a bonus). Students build the other two modules: adapt FinRobot's valuation into a filter that can be backtested point-in-time, and develop a timing module that controls total exposure. All three modules plug into the same backtester, and an ablation table measures how much each one actually contributes.

Selection
Provided · retrainable

Valuation
Built by students

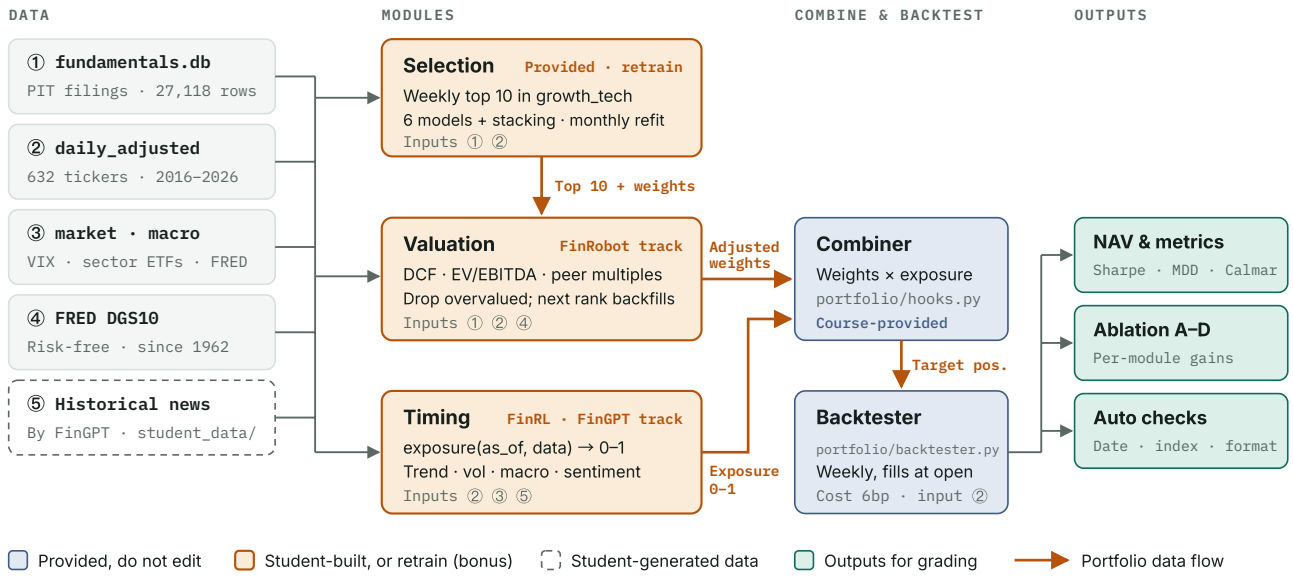
Timing
Built by students

DUE	TEAMS	DATA	VERSION
Wednesday, Nov 25, 2026	1–3 students, one module per team	through 2026-11-06	2026-11-09

AI4Finance Foundation · MA Mentored Research

System architecture

The 5 data sources feed the modules as numbered. Selection is provided, and students can retrain it for a bonus; students build Valuation and Timing, and both hooks are already in place in the backtester.



Code map

The code is in the course repo at `Fall-2026/Assignment2/source_code/`, split into Selection, Valuation and Timing, plus two common directories for portfolio backtesting and shared utilities. Students edit only one file in their own module. The data is distributed as a separate bundle and is not in git.

The folder students receive

```
source_code/
├── selection/                Selection: course model, full source readable  Provided · retrainable
│   ├── selector.py           Model: features, 6 models + stacking, monthly training
│   ├── strategy.py           Monthly predictions → ranking → target_weights()
│   ├── allocation.py  ranking.py  config.py  configs/
│   ├── scores.py             Reads precomputed monthly scores; backtest in seconds
│   ├── build_scores.py       Retrains every month to regenerate scores, ~1.5 h
│   └── run_picks.py          Prints target holdings for a given date
├── valuation/                Valuation: valuation teams
│   ├── student_valuation.py  value() + filter_candidates()  Students write
│   ├── examples.py           Two examples: trailing P/E and per-share DCF  Adapt
│   └── finrobot/             FinRobot DCF engine, Apache-2.0  Do not edit
├── timing/                   Timing: timing teams
│   ├── student_timing.py     exposure(as_of, data) → 0–1  Students write
│   ├── baselines.py          full / sma200 / engine
│   └── engine.py             6-factor risk engine, source of the engine baseline  Must read
├── portfolio/                Portfolio and backtest  Provided
```

```
│   ├── backtester.py          Weekly rebalance at open; lines 137–142 hook in Valuation and Timing  Must read
│   └── hooks.py               Wires student functions into the backtester and checks outputs
├── shared/                    Shared  Provided
│   ├── data_view.py           DataView: the only data access for student functions  Must read
│   ├── guard.py               File reads or network calls in student functions raise errors
│   ├── store.py  prices.py  universe.py   Fundamentals DB, daily prices, index members
│   └── metrics.py  periods.py
├── run_portfolio.py           Portfolio backtest; --ablation runs A/B/C/D at once
├── evaluate_valuation.py      Valuation rank IC and interval coverage
├── evaluate_timing.py         Tests Timing on SPY only, vs. 3 baselines
├── check_submission.py        Pre-submission automated checks
├── student_data/              Student-generated daily data, e.g. news sentiment  Students add
└── data/                      Course data bundle, distributed separately  Not in git
```

What is in the data/ bundle

File	Contents	Through
fundamentals.db	① Quarterly filings (ratios and per-share values), index member snapshots, ticker changes	Filed through 2026-11-06
prices/daily_adjusted.parquet	② Adjusted daily prices for 632 stocks plus SPY and QQQ	2026-11-06
prices/market_series.parquet	③ SPY, ^VIX, 11 sector ETFs	2026-11-06
macro/	③④ DGS10, T10Y2Y, BAA10Y, BAMLH0A0HYM2	2026-11-05/06
selection/monthly_scores.parquet	Monthly selection scores, 2017-01 to 2026-11	2026-11
DATA_MANIFEST.json	Source and cutoff date for each dataset	

Getting started in week 1

1

Set up the environment and data

Python 3.11 or later. Download the data bundle (29 MB) from Dropbox, unzip it inside `source_code/` as `source_code/data/`, then run from the `source_code/` directory:

```
pip install -r requirements.txt
python run_portfolio.py --period dev
```

This is the group A result (Selection only), in about 15 seconds: total return 226.0%, Sharpe 0.98, max drawdown −33.0%. If your numbers match, the environment is set up correctly.

2 **Understand the weekly loop**

Open `portfolio/backtester.py`, lines 128–145. Every Monday it does three things: line 137 computes the selection weights, lines 140–142 multiply total exposure by the Timing output, and orders are placed at the open. The Valuation module lives in `portfolio/hooks.py` and runs before the weights are computed.

3 **First confirm the do-nothing version**

The initial versions of both student files do nothing. Run `python run_portfolio.py --period dev --ablation --timing student`; groups A, B, C and D must give identical results, otherwise something is wrong with your environment.

4 **Then add logic step by step**

After each change, run on the dev period 2017–2021 and record the result. Choose the final configuration only on the val period, and run the test period once at the end. Every test-period run is logged to `output/test_runs.jsonl`.

Interfaces of the two student files

```
# valuation/student_valuation.py
def value(as_of, tickers, data) ->
pd.DataFrame:
    """Valuation for each stock.
    Returns columns: ticker, fair_value,
low, high,
            mispricing, method_spread
    mispricing = fair_value / price - 1"""

def filter_candidates(as_of, candidates,
data):
    """Remove stocks you judge overvalued;
    the next-ranked name backfills. Remove
only, never modify."""
    return candidates
```

```
# timing/student_timing.py
def exposure(as_of, data) -> float:
    """Return total exposure in 0.0–1.0.
    The backtester then applies a 0.95
cash buffer."""
    return 1.0
```

```
# data is the DataView from
shared/data_view.py
data.close(["SPY"], lookback=252)
data.market("^VIX")
data.macro("DGS10")
data.fundamentals(tickers)
data.members()
data.extra("news_sentiment")
```

`data` returns only data on or before `as_of`. The three fundamentals columns that would be known only after the decision date (`trade_price`, `adj_close_q`, `y_return`) have been removed. While student functions run, file reads and network calls raise errors, so look-ahead bias is blocked at the code level.

```
# Common commands
python run_portfolio.py --period dev --ablation --timing student    # A/B/C/D ablation
python evaluate_valuation.py --period dev                           # Valuation rank IC
python evaluate_timing.py --period dev                              # Timing vs. 3 baselines
python check_submission.py                                          # Pre-submission checks
```

Valuation teams: what in FinRobot is usable

Four directly callable files are already in `valuation/finrobot/`, with only the import lines changed. Links in the table go to the exact line on GitHub.

Location	What it is	How to use
finrobot/dcf.py:48	`calculate_dcf(inputs)`: given a set of inputs, computes per-share value `implied_price`	Call directly
finrobot/dcf.py:177	`calculate_sensitivity`: WACC × terminal growth grid, used for low and high	Call directly
finrobot/financial.py:472	`DCFInputs`: the 15 DCF inputs; the table below shows where each comes from	**You fill in**
finrobot/wacc.py:25	`adjust_beta_blume`: adjusts a regression beta	Call directly
examples.py	`trailing_pe_value` (peer median P/E) and `per_share_dcf` (calls the DCF above)	Adapt
historical_valuation.py:85	`compute_historical_band`: P25/P50/P75 of the stock's own historical multiples	Reimplement logic
valuation_aggregator.py:999	EV/EBITDA method: own-history band × TTM EBITDA − net debt	Reimplement logic
valuation_synthesis.py:730	`verdict_from_upside`: how undervalued counts as BUY, how overvalued as SELL	Reference thresholds
dcf_seed.py:43–59	Rate 4.3% and ERP 4.23% hard-coded to 2026 values	Do not use
pipelines/dcf.py:33	`get_forward_revenue_growth`: current FMP consensus	Do not use
valuation_aggregator.py:514	Forward P/E peer method, using current forward EPS	Do not use; use trailing

Where the DCF inputs come from

The fundamentals DB has only ratios and per-share values, with no total revenue, total debt or share count, so everything is computed per share: set `shares_outstanding` to 1 and `implied_price` is the per-share value. Per-share values are single-quarter; for TTM, sum the latest 4 quarters. `per_share_dcf` in `valuation/examples.py` follows this table.

DCFInputs field	Data source	Notes
revenue_base	revenue_per_share	Sum of latest 4 quarters

DCFInputs field	Data source	Notes
`revenue_growth_rates`	`revenue_per_share`	Your own estimate: historical YoY or CAGR, decaying yearly to terminal growth. The most important assumption
`ebitda_margin`	`ebitda_margin`	Median of latest 4 quarters
`da_pct_revenue`	`ebitda_margin − operating_margin`	No D&A in the DB; derived from the difference of the two margins
`capex_pct_revenue`	`capex_per_share ÷ revenue_per_share`	Sum 4 quarters, then divide
`nwc_pct_revenue`	`Not in the DB`	Set to 0 or a fixed value; state it in the report
`tax_rate`	`effective_tax_rate`	Clip to 0–35%
`risk_free_rate`	`data.macro("DGS10")`	10-year Treasury yield on as_of, divided by 100
`beta`	`data.close([t, "SPY"])`	Regress 5 years of weekly returns on SPY, then apply `adjust_beta_blume`
`equity_risk_premium`	`Fixed value`	E.g. 5%; state it in the report
`cost_of_debt`	`DGS10 + fixed spread`	E.g. plus 2%
`debt_ratio`	`debt_to_capital`	Latest quarter
`terminal_growth_rate`	`Fixed value`	2.5%–3%
`shares_outstanding`	`1`	Per-share basis
`net_debt`	`interest_debt_per_share − cash_per_share`	Net debt per share; negative means net cash

Timing teams: code to read

Location	What it is	How to use
`portfolio/backtester.py:140–142`	Timing hook: total exposure = 0.95 × your return value	Must read
`timing/baselines.py`	Three baselines (full, sma200, engine), written the same way as your function	Must read
`timing/engine.py:507`	`exposure_from_history`: entry point of the engine baseline	Understand, then modify

Location	What it is	How to use
`timing/engine.py:59–67`	Weights of the 6 factors and the 16% volatility target	Understand, then modify
`timing/engine.py:318–367`	How each factor is scored	Understand, then modify
`portfolio/backtester.py:38–53`	Stop parameters: 10-day high, 15% stop, 5-day cooldown; enable with `--stops`	**Tunable**
`timing/engine.py:407–468`	Live news sentiment: calls FMP and OpenAI, only gets today's news	Not usable in backtests

Available data

Call	Contents	Notes
`data.close(["SPY"])`	`Adjusted daily prices for SPY, QQQ and 632 stocks`	From 2016-06
`data.market("^VIX")`	`VIX, SPY, 11 sector ETFs`	XLC listed only in 2018-06
`data.macro("T10Y2Y")`	`10-year minus 2-year term spread`	From 1976
`data.macro("BAA10Y")`	`BAA corporate minus 10-year Treasury`	From 1986; use it in place of the high-yield spread
`data.macro("BAMLH0A0HYM2")`	`High-yield spread`	FRED provides only the last 3 years, from 2023-09
`data.extra("news_sentiment")`	`Daily sentiment scores generated by FinGPT teams`	Put in `student_data/`; must have a date column giving the day it became known

Selection: readable and rerunnable

Location	What it is	How to use
`selection/strategy.py:113`	`target_weights`: from scores to per-stock weights	Must read (Valuation)
`selection/strategy.py:62`	`Prediction`: symbol, bucket, rank_score, weight_score	Must read (Valuation)
`selection/selector.py:463`	`run_monthly`: main loop that trains the 6 models each month	Optional

Location	What it is	How to use
`selection/selector.py:1237`	`_clip_trade_prices_to_as_of`: how look-ahead bias is avoided, carried over from Assignment 1	Optional
`selection/scores.py`	The backtest reads precomputed scores, identical to live monthly training	Provided
`selection/build_scores.py`	Regenerates scores after model changes, ~1.5 h; must save separately with `--out`, then backtest with `run_portfolio.py --scores`	**Bonus only**

Bonus: retrain the selection model. For example, fix `incumbent_multiplier`, which has no actual effect (see the comment at line 129 of `strategy.py`), or the winsorization quantiles, which are computed over the full sample.

Most common mistakes

- **Do not use the DB's `pe` and `ev_multiple` columns directly.** Their definitions are unverified: NVDA's 2026-09-30 row has pe 19.9, while price over the latest 4 quarters of EPS is about 28.7. Compute from EPS and price yourself; `check_submission.py` checks this.
- **Do not read files or access the network in student functions.** It raises an error. Put your own generated data in `student_data/` and read it with `data.extra()`.
- **Do not use FinRobot's default rates and consensus estimates.** They are 2026 values; using them in 2018 means knowing the future.
- **Do not have an LLM value or score historical dates.** Its training data already contains later prices and news.
- **Do not tune outside the dev period.** 2022–2023 is only for choosing the final configuration; 2024 onward is run once.

Code and data review

Before designing the assignment, I reviewed the selection code, the FinRobot valuation code and the data. The selection code can be used as is. The FinRobot valuation code is usable, but it is written for "today", so backtesting it directly introduces look-ahead bias. The data has been refreshed through 2026-11-06, and two issues were fixed.

Course selection model `selection/`

`Dev return +226.0%` `Sharpe 0.98`

`Vol 28.3%` `Max DD −33.0%`

- 6 models plus stacking, retrained monthly, rebalanced weekly, holding the top 10 in growth_tech.
- Good point-in-time handling: index members by date, fundamentals by filing date, momentum prices truncated at the decision date.
- The backtester already has two hooks: weights at line 137 and total exposure at lines 140–142. Valuation and Timing plug in here.
- The four numbers above are dev-period (2017–2021) results on the new data. That is nearly 100 points more than SPY (+130.4%), but with the same Sharpe as SPY, and it trails QQQ (+245.6%, Sharpe 1.23).
- In 2026 (01-02 to 11-06): return −2.7%, max drawdown −45.2%, vs. SPY +8.5%. High volatility and deep drawdowns mean the Timing module has real work to do.
- The blacklist has been cleared: the original list was chosen after the fact and applied to the whole backtest, which is look-ahead. Dev-period results are unaffected.

FinRobot valuation `valuation/finrobot/`

`DCF` `Forward P/E peers`

`EV/EBITDA own band`

- Combines three methods in a football field; when they disagree too much (e.g. 2.6×), it gives only a range, no point estimate.
- DCF is now deterministic: inputs are 3-year medians from filings; the LLM only writes the explanation and picks no numbers.
- Risk-free rate 4.3% and ERP 4.23% are hard-coded to January 2026 values.
- DCF growth and the P/E peer method use the **current** FMP consensus, and peers are also screened as of today; none of this is available historically.
- The DCF range is a fixed ±20%, not a real distribution.

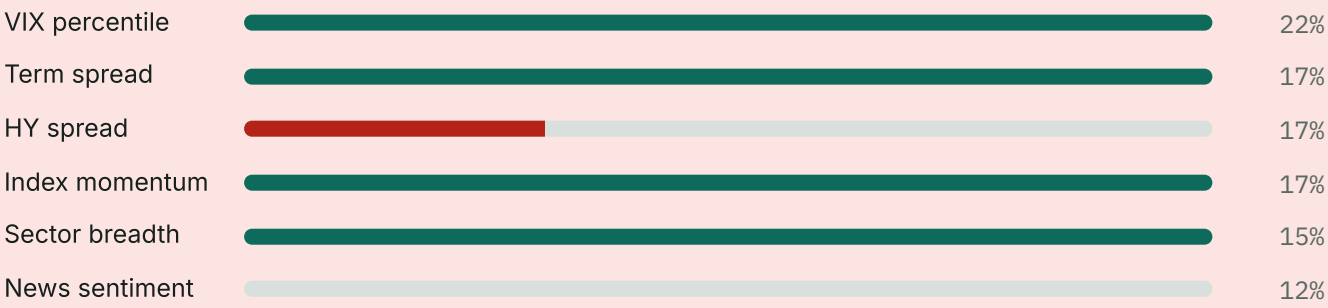
Data refresh `as of 2026-11-06`

- Filings through 11-06, index member snapshots through 11-09; daily prices, VIX and sector ETFs through 11-06; FRED through 11-05/06.
- Added DGS10 (10-year Treasury, the DCF risk-free rate) and BAA10Y (credit spread from 1986).
- All daily prices re-downloaded: the old data had an adjustment seam on 2025-06-02, and the CTVA split was unadjusted, producing a fake −84% crash. Now fixed.

Validation results

- Timing baselines (dev period, SPY only): fully invested Sharpe 0.98, drawdown −33.7%; 200-day MA 1.07, −15.9%; engine 1.18, −13.7%.
- In March 2020 both timing baselines avoided about 20% of the decline but missed about 42% of the rebound.
- The P/E valuation example has a 63-day rank IC of −0.17 (t = −3.2) in 2017–2019 and −0.13 (t = −2.7) over the whole dev period: growth stocks that look cheap on P/E actually underperform. This is exactly what Valuation teams should investigate.
- Of the 119 months of precomputed selection scores, spot checks of 2019-05 and 2019-06 retrained live matched exactly.
- In a fresh Python 3.14 environment, downloading data from Dropbox, installing dependencies and running every command gave `check_submission.py` 0 FAIL.
- Applied to the stock portfolio (2017–2019), the 200-day MA made max drawdown worse, from −28% to −39%: it whipsawed in and out in Oct–Dec 2018 and only re-entered in Feb 2019, missing the rebound.

Only 4 factors are active in the engine timing model. `timing/engine.py` computes a risk score from 6 factors, then scales by a 16% volatility target. The cause is known: FRED publishes only the last 3 years of the high-yield spread, so before 2023-09 that factor takes a neutral value; historical news sentiment is always fixed at 0.5. Timing teams can replace the former with BAA10Y, and FinGPT teams can fill in the latter.



Bar length is the share of the 2017–2026 backtest with valid data; the right column is the factor weight.

Selection

`Provided · students can retrain (bonus)`

Shared by everyone; retraining it earns a bonus

What students get

- Precomputed monthly scores for 2017-01 to 2026-11 in `data/selection/monthly_scores.parquet`.
- The full source of the selection model in `selection/`, readable, and retrainable to your own file with `build_scores.py --out` (bonus).

Why it is precomputed

- Retraining monthly over all 119 months takes about 1.5 hours even with 4 parallel processes; with precomputed scores, one dev-period backtest takes about 15 seconds.
- With a fixed list, everyone's Valuation and Timing results are comparable.

> **Bonus: retrain the selection model.** For example, fix `incumbent_multiplier` having no actual effect, or winsorization quantiles computed over the full sample. You must save with `--out` and not overwrite the course scores. You must also retrain the original `growth.json` in the same environment as a control, because different scikit-learn and xgboost versions change the original model's scores too (rank correlation 0.88–0.96).

Valuation: overvalued or undervalued `Built by students`

Mainly the FinRobot track

Can the three methods be computed point-in-time?

Method	Current state	Backtestable as is?	What students must change
EV/EBITDA own band	Own 5-year multiple P25–P75 × TTM EBITDA	**Mostly**	Pull data from `fundamentals.db` by filing date
DCF	3-year median inputs; growth prefers consensus; rates hard-coded to 2026	**Partly**	Use the then-current 10-year Treasury (FRED DGS10) for rates; use historical CAGR or your own forecast for growth; make the range a real sensitivity distribution
Forward P/E peers	Current FMP forward EPS × current peer median	**No**	Switch to trailing P/E; peers become index members in the same GICS sector at that time

Output format

```
value(as_of, tickers, data) returns:
  ticker, fair_value, low, high,
  mispricing     # fair_value / price - 1
  method_spread  # max / min across
methods
```

How it enters the portfolio

- Design rules in `filter_candidates`, e.g. drop names overvalued by more than X%, with the 11th-ranked name backfilling automatically.
- You must explain how you handle large disagreement among the three methods.

How it is evaluated

- Rank IC of mispricing against forward 3-, 6- and 12-month returns.
- Whether the share of prices falling in [low, high] matches the claimed confidence level.
- Change in Sharpe, drawdown and turnover once added to the portfolio.

What to tell students up front

- The growth_tech top 10 are almost all "expensive", so a valuation filter may well hurt returns. Honestly reported negative results earn full credit.
- LLMs may not be used to value historical dates: their training data already contains later prices.

Timing: control exposure and when to trade

`Built by students`

Mainly the FinRL and FinGPT tracks

Interface

```
def exposure(as_of, data) -> float:
    # data only provides data on or before
as_of
    # returns 0.0–1.0, multiplied into
total exposure
    ...
```

Per-stock stops are optional: the backtester already has two 15% stop layers (from the 10-day high and from the entry price). You may tune the parameters or replace the rules.

Three baselines, all required

1. Fully invested, no timing.
2. SPY 200-day moving average rule.
3. The 6-factor risk engine in `timing/engine.py`, with an audit of its missing data.

Usable signals

- Trend and momentum, volatility targeting, drawdown control.
- Macro: term spread, BAA10Y credit spread, VIX.
- Historical news sentiment generated with FinGPT.
- FinRL reinforcement learning agents.

How it is evaluated

- Test on SPY alone first, then on the selection portfolio.
- Sharpe, max drawdown, Calmar, average exposure, number of rebalances.
- Three drawdowns (2018Q4, 2020-03, all of 2022): how much of each decline was avoided, and how much of the rebound was missed.
- Parameter sensitivity: the sample has only three meaningful declines, so rules must be fixed in advance.

How the three tracks divide the work

Teams of 1–3; each team owns one module and submits one assignment, using baselines for the other two modules. At the end, the best Valuation and the best Timing can be combined into a class portfolio.

FinRL-Trading

Timing

Exposure control is a classic reinforcement learning problem; a rule-based version is also fine.

FinRobot

Valuation

Adapt `finrobot_desktop` directly; strong work can be submitted upstream as a PR.

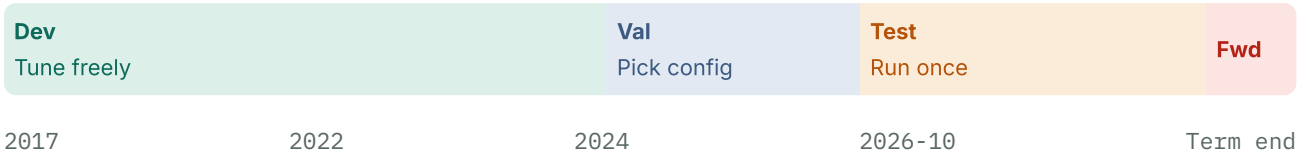
FinGPT

Timing · sentiment factor

Fill in the 12% in `timing/engine.py` that is stuck at 0.5: score historical news, build a daily sentiment series, and show that it improves Timing.

Evaluation protocol

In Assignment 1, several students chose strategies after seeing results, and there was no holdout set. This time the split is fixed, and the test period can be run only once.



2017 – 2021
Tune and experiment freely.

2022 – 2023
Fix the final configuration here and preregister it by submitting the commit hash.

2024 – 2026-10-09
Run once; the report states which run number it is.

From 2026-11
You run everyone's code on updated data (optional).

Ablation table (required)

Group	Selection	Valuation	Timing	Question answered
A	✓	—	Full	Performance of Selection alone
B	✓	✓	Full	Marginal contribution of the valuation filter
C	✓	—	✓	Marginal contribution of Timing
D	✓	✓	✓	Whether the two add up or cancel out

Lessons carried over from Assignment 1

- [ ] Reproducible with one command on a clean clone, using only course data.
- [ ] Automated check script: no future dates, universe is the then-current index members, correct output format. Failing submissions are returned.
- [ ] AI use is allowed, but the "Judgment and Reflection" section must be written by you; leftover agent text loses points.
- [ ] The Medium link must be published.

Draft grading rubric

100 points total. Test-period performance is deliberately worth only 5 points, so students do not gamble on returns.

Module implementation and soundness Is the design justified, and is the code correct	25
Single-module evaluation Compared against all baselines, complete metrics, numbers match files	20
Bias and data audit Point-in-time, missing data, LLM look-ahead bias	15
Ablation table and marginal contribution Groups A/B/C/D, explaining whether effects add up or cancel	15
Test-period performance Stability relative to baselines, not absolute returns	5
Report and personal judgment Written by you, negative results reported honestly	10
Reproducibility and submission One-command reproduction, passes automated checks	10

STAT GR5398 MA Mentored Research · Fall 2026 · Assignment 2 design draft. Based on: the course selection model (data through 2026-11-06) and the FinRobot finrobot_desktop valuation source.